Exercices - Série 1, parties 1 et 2

420-249-AH - Programmation mobile iOS

1. Exercice 1 - Analyse des températures d'une semaine

Une application météo reçoit les températures maximales de plusieurs jours. Certaines mesures peuvent être absentes à cause d'un problème de capteur.

On vous donne les données suivantes :

```swift
let temperatures: [Double?] = [
    18.5, 21.0, nil, 16.5,
    24.0, nil, 19.5
]

let jours = [
    "Lundi", "Mardi", "Mercredi",
    "Jeudi", "Vendredi", "Samedi",
    "Dimanche"
]
```

Vous devez réaliser un programme Swift qui respecte les exigences suivantes.

1. Analyser une température

Créer une fonction :

```swift
func analyserTemperature(
    _ temperature: Double?
) -> String
```

La fonction doit retourner :

Température	Résultat
`Absente`	"Donnée indisponible"
`Inférieure à 15`	"Froid"
`De 15 à 20 inclusivement`	"Doux"
`Supérieure à 20`	"Chaud"

2. Afficher chaque jour

Parcourir les températures de la semaine et afficher pour chaque jour un résultat semblable à :

```
Console
Lundi : 18.5°C - Doux
Mardi : 21.0°C - Chaud
Mercredi : Donnée indisponible
```

3. Compter et comparer

Pendant le parcours, calculer :

- Le nombre de températures réellement disponibles
- Le nombre de mesures absentes
- La température la plus élevée
- La température la plus basse

4. Calculer la moyenne

Calculer la moyenne de la semaine en ignorant les valeurs nil.

5. Jours chauds

Créer un tableau nommé :

```
var joursChauds: [String]
```

qui contiendra uniquement les noms des jours dont la température est supérieure à 20°C.

6. Afficher le résumé

À la fin du programme, afficher un résumé similaire à :

```
Console
Nombre de mesures disponibles : ...
Nombre de mesures absentes : ...
Température minimale : ...
Température maximale : ...
Température moyenne : ...
Jours chauds : ...
```

7. Cas particulier

Si toutes les températures du tableau sont nil, le programme ne doit pas tenter de calculer une moyenne.

Il doit afficher :

```
Console
Impossible de calculer la moyenne.
```

2. Exercice 2 - Gestion simplifiée d'un panier d'achat

Une application mobile doit calculer le montant d'un panier.

Chaque produit est représenté par un tuple contenant :

- Son nom
- Son prix
- Sa quantité disponible

Utilisez les données suivantes :

```
let produits = [
    (nom: "Clavier", prix: 79.99, stock: 4),
    (nom: "Souris", prix: 39.99, stock: 0),
    (nom: "Écran", prix: 249.99, stock: 2),
    (nom: "Webcam", prix: 89.99, stock: 5),
    (nom: "Casque", prix: 129.99, stock: 1)
]
```

Le client souhaite acheter les produits suivants :

```
let panier = [
    "Clavier": 1,
    "Souris": 1,
    "Webcam": 2,
    "Casque": 3
]
```

Le dictionnaire représente :

```
Nom du produit → quantité demandée
```

Votre programme doit traiter chaque élément du panier.

1. Trouver le prix d'un produit

Créer une fonction :

```
func trouverPrix(
    du produit: String,
    dans produits: [(nom: String, prix: Double, stock: Int)]
) -> Double?
```

Cette fonction doit retourner le prix du produit s'il existe dans la collection, sinon nil.

2. Trouver le stock d'un produit

Créer une deuxième fonction permettant de retrouver le stock d'un produit :

```
func trouverStock(
    du produit: String,
    dans produits: [(nom: String, prix: Double, stock: Int)]
) -> Int?
```

3. Vérifier chaque situation

Pour chaque produit demandé dans le panier, vérifier les situations suivantes :

Situation	Message à afficher
`Le produit n'existe pas`	...
`Le produit existe mais son stock est 0`	produit en rupture de stock.
`La quantité demandée dépasse le stock disponible`	quantité insuffisante. Stock disponible : ...
`La quantité demandée est disponible`	commande acceptée.

4. Afficher le message adapté

Exemples de résultats attendus :

```
Console
Clavier : commande acceptée.
Souris : produit en rupture de stock.
Casque : quantité insuffisante. Stock disponible : 1.
```

5. Calculer le sous-total

Lorsqu'une commande est acceptée, calculer son sous-total :

```
prix × quantité
```

et l'ajouter au total du panier.

6. Produits refusés

Conserver les noms des produits dont la commande a été refusée dans :

```
var produitsRefuses: [String] = []
```

7. Total, réduction et taxes

Une fois tous les produits analysés :

- Afficher le total avant taxes
- Appliquer une réduction de 10 % si le total est supérieur ou égal à 200 $
- Calculer les taxes de 14,975 %
- Afficher le total final

8. Fonction de réduction

Créer une fonction :

```
func calculerReduction(
    total: Double
) -> Double
```

qui retourne le montant de la réduction à appliquer.

9. Aucune commande valide

Si aucun produit ne peut être acheté, afficher :

```
Console
Aucune commande valide.
```

et ne pas calculer les taxes.

10. Afficher les produits refusés

À la fin, afficher également la liste des produits refusés, seulement si cette liste n'est pas vide.